

MasterClass in DSA

Exercise Set - 2

Topic: Singly Linked List

Total Exercises: 60

Easy: 20 | Intermediate: 25 | Difficult: 15

CoreCode Programming Academy
An Academy by Yogeshwar Shukla

General Instructions

1. Use the provided list.h and list.c as your base implementation. Add new functions to extend the library.
2. Follow the existing coding conventions: use status_t for return status, data_t for data, len_t for lengths.
3. Return appropriate status codes: SUCCESS, LIST_EMPTY, LIST_DATA_NOT_FOUND as applicable.
4. Test your implementations thoroughly using assertions as demonstrated in use_list.c.
5. Unless specified, do not use additional data structures. Work with pointers and the existing node structure.
6. For functions returning a new list, the original list(s) should remain unchanged (immutable operation).
7. For functions modifying the list in-place, clearly document this behavior (mutable operation).
8. List notation: [START]->[data1]->[data2]->...->[END] represents the linked list structure.

Section A: Easy Exercises

Exercise 1: Sum of All Elements EASY

Write a function that returns the sum of all data values in the list. Return 0 if the list is empty.

```
data_t get_sum(list_t* p_list)
```

Sample Input:

[START]->[4]->[7]->[2]->[9]->[END]

Sample Output:

22

Exercise 2: Count Nodes Greater Than X EASY

Write a function that counts and returns the number of nodes whose data value is strictly greater than the given value x.

```
len_t count_greater(list_t* p_list, data_t x)
```

Sample Input:

[START]->[10]->[5]->[20]->[3]->[15]->[END], x = 8

Sample Output:

3

Exercise 3: Count Nodes Less Than X EASY

Write a function that counts and returns the number of nodes whose data value is strictly less than the given value x.

```
len_t count_less(list_t* p_list, data_t x)
```

Sample Input:

[START]->[10]->[5]->[20]->[3]->[15]->[END], x = 12

Sample Output:

3

Exercise 4: Find Maximum Element EASY

Write a function that finds the maximum data value in the list. Return LIST_EMPTY if list is empty, otherwise store the maximum in *p_max and return SUCCESS.

```
status_t get_max(list_t* p_list, data_t* p_max)
```

Sample Input:

[START]->[45]->[12]->[78]->[34]->[END]

Sample Output:

*p_max = 78, returns SUCCESS

Exercise 5: Find Minimum Element EASY

Write a function that finds the minimum data value in the list. Return LIST_EMPTY if list is empty, otherwise store the minimum in *p_min and return SUCCESS.

```
status_t get_min(list_t* p_list, data_t* p_min)
```

Sample Input:

[START]->[45]->[12]->[78]->[34]->[END]

Sample Output:

*p_min = 12, returns SUCCESS

Exercise 6: Count Even Numbers EASY

Write a function that counts how many nodes contain even data values.

```
len_t count_even(list_t* p_list)
```

Sample Input:

[START]->[3]->[8]->[12]->[5]->[7]->[10]->[END]

Sample Output:

3

Exercise 7: Count Odd Numbers EASY

Write a function that counts how many nodes contain odd data values.

```
len_t count_odd(list_t* p_list)
```

Sample Input:

[START]->[3]->[8]->[12]->[5]->[7]->[10]->[END]

Sample Output:

3

Exercise 8: Get Data at Index EASY

Write a function that retrieves the data at a given 0-based index. Return LIST_DATA_NOT_FOUND if index is invalid.

```
status_t get_data_at_index(list_t* p_list, int index, data_t* p_data)
```

Sample Input:

[START]->[10]->[20]->[30]->[40]->[END], index = 2

Sample Output:

*p_data = 30, returns SUCCESS

Exercise 9: Count Occurrences EASY

Write a function that counts how many times a specific value appears in the list.

```
len_t count_occurrences(list_t* p_list, data_t value)
```

Sample Input:

```
[START]->[5]->[3]->[5]->[7]->[5]->[END], value = 5
```

Sample Output:

3

Exercise 10: Check All Positive EASY

Write a function that returns TRUE if all elements in the list are positive (greater than 0), FALSE otherwise. An empty list should return TRUE.

```
status_t is_all_positive(list_t* p_list)
```

Sample Input:

```
[START]->[5]->[3]->[8]->[1]->[END]
```

Sample Output:

TRUE

Exercise 11: Check All Negative EASY

Write a function that returns TRUE if all elements in the list are negative (less than 0), FALSE otherwise. An empty list should return TRUE.

```
status_t is_all_negative(list_t* p_list)
```

Sample Input:

```
[START]->[-5]->[-3]->[-8]->[END]
```

Sample Output:

```
TRUE
```

Exercise 12: Find First Index of Value EASY

Write a function that returns the 0-based index of the first occurrence of a value. Return -1 if the value is not found.

```
int get_first_index(list_t* p_list, data_t value)
```

Sample Input:

```
[START]->[10]->[20]->[30]->[20]->[END], value = 20
```

Sample Output:

```
1
```

Exercise 13: Find Last Index of Value EASY

Write a function that returns the 0-based index of the last occurrence of a value. Return -1 if the value is not found.

```
int get_last_index(list_t* p_list, data_t value)
```

Sample Input:

```
[START]->[10]->[20]->[30]->[20]->[END], value = 20
```

Sample Output:

```
3
```

Exercise 14: Calculate Average EASY

Write a function that calculates and returns the average of all elements. Return 0.0 if the list is empty.

```
double get_average(list_t* p_list)
```

Sample Input:

[START]->[10]->[20]->[30]->[40]->[END]

Sample Output:

25.0

Exercise 15: Print List in Reverse (Recursive) EASY

Write a function that prints all elements of the list in reverse order using recursion. Do not modify the list.

```
void print_reverse(list_t* p_list)
```

Sample Input:

[START]->[1]->[2]->[3]->[4]->[END]

Sample Output:

4 3 2 1

Exercise 16: Check if Sorted (Ascending) EASY

Write a function that returns TRUE if the list is sorted in non-decreasing order, FALSE otherwise. An empty or single-element list is considered sorted.

```
status_t is_sorted_ascending(list_t* p_list)
```

Sample Input:

```
[START]->[1]->[3]->[5]->[7]->[END]
```

Sample Output:

```
TRUE
```

Exercise 17: Check if Sorted (Descending) EASY

Write a function that returns TRUE if the list is sorted in non-increasing order, FALSE otherwise. An empty or single-element list is considered sorted.

```
status_t is_sorted_descending(list_t* p_list)
```

Sample Input:

```
[START]->[9]->[7]->[5]->[3]->[END]
```

Sample Output:

```
TRUE
```

Exercise 18: Get Middle Element EASY

Write a function that finds the middle element of the list. If the list has even number of elements, return the first of the two middle elements. Return LIST_EMPTY if list is empty.

```
status_t get_middle(list_t* p_list, data_t* p_middle)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[4]->[5]->[END]
```

Sample Output:

```
*p_middle = 3, returns SUCCESS
```

Exercise 19: Count Nodes in Range EASY

Write a function that counts nodes with data values in the range [low, high] (inclusive).

```
len_t count_in_range(list_t* p_list, data_t low, data_t high)
```

Sample Input:

[START]->[5]->[12]->[8]->[3]->[15]->[7]->[END], low = 5, high = 10

Sample Output:

3 (values: 5, 8, 7)

Exercise 20: Check Lists Equality EASY

Write a function that checks if two lists are equal (same length and same data at each position).

```
status_t are_lists_equal(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [START]->[1]->[2]->[3]->[END], List2: [START]->[1]->[2]->[3]->[END]

Sample Output:

TRUE

Section B: Intermediate Exercises

Exercise 21: Insert at Position INTERMEDIATE

Write a function that inserts a new node at the given 0-based position. Position 0 means insert at start. Return LIST_DATA_NOT_FOUND if position is negative or greater than list length.

```
status_t insert_at_position(list_t* p_list, data_t new_data, int position)
```

Sample Input:

```
[START]->[10]->[20]->[30]->[END], new_data = 15, position = 1
```

Sample Output:

```
[START]->[10]->[15]->[20]->[30]->[END], returns SUCCESS
```

Exercise 22: Delete at Position INTERMEDIATE

Write a function that deletes the node at the given 0-based position. Return LIST_EMPTY if list is empty, LIST_DATA_NOT_FOUND if position is invalid.

```
status_t delete_at_position(list_t* p_list, int position)
```

Sample Input:

```
[START]->[10]->[20]->[30]->[40]->[END], position = 2
```

Sample Output:

```
[START]->[10]->[20]->[40]->[END], returns SUCCESS
```

Exercise 23: Insert in Sorted Order INTERMEDIATE

Write a function that inserts a new element into an already sorted list (ascending) such that the list remains sorted after insertion.

```
status_t insert_sorted(list_t* p_list, data_t new_data)
```

Sample Input:

```
[START]->[10]->[20]->[40]->[50]->[END], new_data = 35
```

Sample Output:

```
[START]->[10]->[20]->[35]->[40]->[50]->[END]
```

Exercise 24: Delete All Occurrences INTERMEDIATE

Write a function that removes all nodes containing the specified data value. Return the count of nodes deleted.

```
len_t delete_all_occurrences(list_t* p_list, data_t value)
```

Sample Input:

```
[START]->[5]->[3]->[5]->[7]->[5]->[END], value = 5
```

Sample Output:

```
[START]->[3]->[7]->[END], returns 3
```

Exercise 25: Get Nth Node from End INTERMEDIATE

Write a function that returns the data of the Nth node from the end (1-based). Return LIST_DATA_NOT_FOUND if N is invalid.

```
status_t get_nth_from_end(list_t* p_list, int n, data_t* p_data)
```

Sample Input:

```
[START]->[10]->[20]->[30]->[40]->[50]->[END], n = 2
```

Sample Output:

```
*p_data = 40, returns SUCCESS
```

Exercise 26: Swap Adjacent Nodes INTERMEDIATE

Write a function that swaps every two adjacent nodes. If odd number of nodes, the last node remains in place. Swap nodes, not just data values.

```
status_t swap_pairs(list_t* p_list)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[4]->[5]->[END]
```

Sample Output:

```
[START]->[2]->[1]->[4]->[3]->[5]->[END]
```

Exercise 27: Split List at Position INTERMEDIATE

Write a function that splits a list into two lists at a given position. The original list contains elements before position, the new list contains elements from position onwards.

```
list_t* split_at_position(list_t* p_list, int position)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[4]->[5]->[END], position = 2
```

Sample Output:

```
Original: [START]->[1]->[2]->[END], New: [START]->[3]->[4]->[5]->[END]
```

Exercise 28: Increment All Elements INTERMEDIATE

Write a function that adds a constant value to all elements in the list.

```
status_t increment_all(list_t* p_list, data_t increment)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[END], increment = 10
```

Sample Output:

```
[START]->[11]->[12]->[13]->[END]
```

Exercise 29: Multiply All Elements **INTERMEDIATE**

Write a function that multiplies all elements by a constant factor.

```
status_t multiply_all(list_t* p_list, data_t factor)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[END], factor = 5
```

Sample Output:

```
[START]->[5]->[10]->[15]->[END]
```

Exercise 30: Remove Nodes Greater Than X INTERMEDIATE

Write a function that removes all nodes with data values greater than x. Return the count of nodes removed.

```
len_t remove_greater_than(list_t* p_list, data_t x)
```

Sample Input:

```
[START]->[5]->[15]->[8]->[20]->[3]->[END], x = 10
```

Sample Output:

```
[START]->[5]->[8]->[3]->[END], returns 2
```

Exercise 31: Remove Nodes Less Than X INTERMEDIATE

Write a function that removes all nodes with data values less than x. Return the count of nodes removed.

```
len_t remove_less_than(list_t* p_list, data_t x)
```

Sample Input:

```
[START]->[5]->[15]->[8]->[20]->[3]->[END], x = 10
```

Sample Output:

```
[START]->[15]->[20]->[END], returns 3
```

Exercise 32: Get Sublist INTERMEDIATE

Write a function that returns a new list containing elements from index 'start' to index 'end' (inclusive, 0-based). Return NULL if indices are invalid.

```
list_t* get_sublist(list_t* p_list, int start, int end)
```

Sample Input:

```
[START]->[10]->[20]->[30]->[40]->[50]->[END], start = 1, end = 3
```

Sample Output:

```
New list: [START]->[20]->[30]->[40]->[END]
```

Exercise 33: Replace All Occurrences INTERMEDIATE

Write a function that replaces all occurrences of old_value with new_value. Return the count of replacements made.

```
len_t replace_all(list_t* p_list, data_t old_value, data_t new_value)
```

Sample Input:

```
[START]->[5]->[3]->[5]->[7]->[5]->[END], old_value = 5, new_value = 100
```

Sample Output:

```
[START]->[100]->[3]->[100]->[7]->[100]->[END], returns 3
```

Exercise 34: Rotate List Left by K **INTERMEDIATE**

Write a function that rotates the list to the left by k positions. Handle cases where k is greater than list length.

```
status_t rotate_left(list_t* p_list, int k)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[4]->[5]->[END], k = 2
```

Sample Output:

```
[START]->[3]->[4]->[5]->[1]->[2]->[END]
```

Exercise 35: Rotate List Right by K INTERMEDIATE

Write a function that rotates the list to the right by k positions. Handle cases where k is greater than list length.

```
status_t rotate_right(list_t* p_list, int k)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[4]->[5]->[END], k = 2
```

Sample Output:

```
[START]->[4]->[5]->[1]->[2]->[3]->[END]
```

Exercise 36: Delete Middle Node INTERMEDIATE

Write a function that deletes the middle node of the list. For even-length lists, delete the first of the two middle nodes.

```
status_t delete_middle(list_t* p_list)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[4]->[5]->[END]
```

Sample Output:

```
[START]->[1]->[2]->[4]->[5]->[END]
```

Exercise 37: Separate Even and Odd INTERMEDIATE

Write a function that rearranges the list so all even numbers come before all odd numbers. Maintain the relative order within even and odd groups.

```
status_t separate_even_odd(list_t* p_list)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[4]->[5]->[6]->[END]
```

Sample Output:

```
[START]->[2]->[4]->[6]->[1]->[3]->[5]->[END]
```

Exercise 38: Clone List INTERMEDIATE

Write a function that creates and returns a deep copy of the given list. The new list should have completely independent nodes.

```
list_t* clone_list(list_t* p_list)
```

Sample Input:

[START]->[10]->[20]->[30]->[END]

Sample Output:

New list: [START]->[10]->[20]->[30]->[END] (independent copy)

Exercise 39: Get Union of Two Lists **INTERMEDIATE**

Write a function that returns a new list containing all unique elements from both lists (no duplicates in result).

```
list_t* get_union(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [START]->[1]->[2]->[3]->[END], List2: [START]->[2]->[3]->[4]->[END]

Sample Output:

[START]->[1]->[2]->[3]->[4]->[END]

Exercise 40: Get Intersection of Two Lists INTERMEDIATE

Write a function that returns a new list containing only the common elements present in both lists (no duplicates in result).

```
list_t* get_intersection(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [START]->[1]->[2]->[3]->[4]->[END], List2: [START]->[3]->[4]->[5]->[END]

Sample Output:

[START]->[3]->[4]->[END]

Exercise 41: Get Difference of Two Lists INTERMEDIATE

Write a function that returns a new list containing elements present in the first list but not in the second list.

```
list_t* get_difference(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [START]->[1]->[2]->[3]->[4]->[END], List2: [START]->[3]->[4]->[5]->[END]

Sample Output:

[START]->[1]->[2]->[END]

Exercise 42: Insert List After Position INTERMEDIATE

Write a function that inserts all elements of the second list into the first list after a given position. The second list should become empty after this operation.

```
status_t insert_list_after(list_t* p_list_1, int position, list_t* p_list_2)
```

Sample Input:

List1: [START]->[1]->[2]->[5]->[END], List2: [START]->[3]->[4]->[END], position = 1

Sample Output:

List1: [START]->[1]->[2]->[3]->[4]->[5]->[END], List2: empty

Exercise 43: Find Loop Length (Simulated) INTERMEDIATE

Given a function that can detect if a list has a cycle, write a function that returns the length of the cycle. For this exercise, manually create a cycle by connecting the last node to a node at a given position.

```
len_t get_cycle_length(list_t* p_list)
```

Sample Input:

List with cycle: 1->2->3->4->5->3 (5 connects back to 3)

Sample Output:

3 (cycle: 3->4->5->3)

Exercise 44: Remove Nth Node From End **INTERMEDIATE**

Write a function that removes the Nth node from the end of the list in a single pass.

```
status_t remove_nth_from_end(list_t* p_list, int n)
```

Sample Input:

[START]->[1]->[2]->[3]->[4]->[5]->[END], n = 2

Sample Output:

[START]->[1]->[2]->[3]->[5]->[END]

Exercise 45: Check if Sublist INTERMEDIATE

Write a function that checks if the second list is a contiguous sublist of the first list.

```
status_t is_sublist(list_t* p_list, list_t* p_sublist)
```

Sample Input:

List: [START]->[1]->[2]->[3]->[4]->[5]->[END], Sublist: [START]->[2]->[3]->[4]->[END]

Sample Output:

TRUE

Section C: Difficult Exercises

Exercise 46: Remove All Duplicates DIFFICULT

Write a function that removes all duplicate values from the list, keeping only the first occurrence of each value. Do this without using any additional data structures (use nested traversals).

```
status_t remove_all_duplicates(list_t* p_list)
```

Sample Input:

```
[START]->[5]->[3]->[5]->[7]->[3]->[5]->[8]->[END]
```

Sample Output:

```
[START]->[5]->[3]->[7]->[8]->[END]
```

Exercise 47: Check if Palindrome DIFFICULT

Write a function that checks if the list is a palindrome (reads the same forwards and backwards). An empty list or single-element list is considered a palindrome. Do not use additional data structures except for recursion stack.

```
status_t is_palindrome(list_t* p_list)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[2]->[1]->[END]
```

Sample Output:

```
TRUE
```

Exercise 48: Pairwise Sum List DIFFICULT

Given two lists representing non-negative integers (each node contains a single digit, stored in reverse order), return a new list representing their sum.

```
list_t* add_lists_as_numbers(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [START]->[2]->[4]->[3]->[END] (represents 342), List2: [START]->[5]->[6]->[4]->[END] (represents 465)

Sample Output:

[START]->[7]->[0]->[8]->[END] (represents 807 = 342 + 465)

Exercise 49: Reverse in Groups of K DIFFICULT

Write a function that reverses nodes in groups of K. If remaining nodes are less than K, reverse them as well.

```
status_t reverse_in_groups(list_t* p_list, int k)
```

Sample Input:

[START]->[1]->[2]->[3]->[4]->[5]->[6]->[7]->[8]->[END], k = 3

Sample Output:

[START]->[3]->[2]->[1]->[6]->[5]->[4]->[8]->[7]->[END]

Exercise 50: Merge K Sorted Lists **DIFFICULT**

Write a function that takes an array of K sorted lists and merges them into a single sorted list.

```
list_t* merge_k_sorted_lists(list_t** pp_lists, int k)
```

Sample Input:

Lists: [1->4->7], [2->5->8], [3->6->9]

Sample Output:

[START]->[1]->[2]->[3]->[4]->[5]->[6]->[7]->[8]->[9]->[END]

Exercise 51: Flatten a Multi-Level List **DIFFICULT**

Design and implement a multi-level linked list where each node can have a 'child' pointer to another list. Write a function to flatten it into a single-level list (all child lists appended after their parent node).

```
status_t flatten_list(multilevel_list_t* p_list)
```

Sample Input:

1->2->3->4 (2 has child: 5->6, 3 has child: 7)

Sample Output:

[START]->[1]->[2]->[5]->[6]->[3]->[7]->[4]->[END]

Exercise 52: Partition List Around Value **DIFFICULT**

Write a function that partitions the list around a value x, such that all nodes less than x come before nodes greater than or equal to x. Preserve the original relative order within each partition.

```
status_t partition(list_t* p_list, data_t x)
```

Sample Input:

[START]->[1]->[4]->[3]->[2]->[5]->[2]->[END], x = 3

Sample Output:

[START]->[1]->[2]->[2]->[4]->[3]->[5]->[END]

Exercise 53: Sort List Using Merge Sort **DIFFICULT**

Write a function that sorts the linked list in ascending order using the merge sort algorithm. Do not convert to array.

```
status_t merge_sort_list(list_t* p_list)
```

Sample Input:

[START]->[4]->[2]->[1]->[3]->[END]

Sample Output:

[START]->[1]->[2]->[3]->[4]->[END]

Exercise 54: Detect and Remove Loop **DIFFICULT**

Write a function that detects if a list has a cycle (loop) and removes it, restoring the list to a proper singly linked list. Use Floyd's cycle detection algorithm.

```
status_t detect_and_remove_loop(list_t* p_list)
```

Sample Input:

1->2->3->4->5->3 (5's next points to node containing 3)

Sample Output:

[START]->[1]->[2]->[3]->[4]->[5]->[END] (loop removed)

Exercise 55: Intersection Point of Two Lists **DIFFICULT**

Write a function that finds the node where two lists merge (Y-shaped intersection). The lists may have different lengths before the intersection point.

```
node_t* find_intersection_point(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: 1->2->3->6->7, List2: 4->5->6->7 (merge at node 6)

Sample Output:

Pointer to node containing 6

Exercise 56: Segregate 0s, 1s, and 2s **DIFFICULT**

Write a function that sorts a list containing only 0s, 1s, and 2s such that all 0s come first, then all 1s, then all 2s. Do this by changing links, not data.

```
status_t segregate_012(list_t* p_list)
```

Sample Input:

[START]->[1]->[2]->[0]->[1]->[2]->[0]->[1]->[END]

Sample Output:

[START]->[0]->[0]->[1]->[1]->[1]->[2]->[2]->[END]

Exercise 57: Alternate Split **DIFFICULT**

Write a function that splits a list into two lists by alternating nodes. The first list gets nodes at even indices (0, 2, 4...), the second gets nodes at odd indices (1, 3, 5...). Move actual nodes, don't create new

ones.

```
void alternate_split(list_t* p_source, list_t** pp_list_a, list_t** pp_list_b)
```

Sample Input:

[START]->[1]->[2]->[3]->[4]->[5]->[6]->[END]

Sample Output:

ListA: [START]->[1]->[3]->[5]->[END], ListB: [START]->[2]->[4]->[6]->[END]

Exercise 58: Rearrange List (Fold) **DIFFICULT**

Write a function that rearranges the list by folding: first node, last node, second node, second-last node, and so on. Do this in-place by manipulating pointers.

```
status_t fold_list(list_t* p_list)
```

Sample Input:

```
[START]->[1]->[2]->[3]->[4]->[5]->[6]->[END]
```

Sample Output:

```
[START]->[1]->[6]->[2]->[5]->[3]->[4]->[END]
```

Exercise 59: Delete Nodes with Greater Value on Right **DIFFICULT**

Write a function that deletes all nodes that have a greater value somewhere to their right. The rightmost node is never deleted.

```
status_t delete_nodes_with_greater_on_right(list_t* p_list)
```

Sample Input:

```
[START]->[12]->[15]->[10]->[11]->[5]->[6]->[2]->[3]->[END]
```

Sample Output:

```
[START]->[15]->[11]->[6]->[3]->[END]
```

Exercise 60: Clone List with Random Pointer **DIFFICULT**

Design a node structure with an additional 'random' pointer that can point to any node in the list or NULL. Write a function to create a deep copy of such a list, where the random pointers in the cloned list point to corresponding cloned nodes.

```
random_list_t* clone_random_list(random_list_t* p_list)
```

Sample Input:

```
List where node1.random->node3, node2.random->node1, node3.random->NULL
```

Sample Output:

Cloned list with same structure and random pointer relationships